Class Notes: Experiment 8 - Kafka-Based Event-Driven Pipeline 🚀

Topic: How to build a modern backend that doesn't crash or lose data when things go wrong!

Part 1: All Topic Explanations 🧠

1. What is Event-Driven Architecture (EDA)? 🤔

Usually, when you build an app, Service A directly calls Service B and waits for an answer. (This is called **Synchronous**).

- **The Problem:** If Service B is slow or dead, Service A gets stuck waiting!
- **The Solution (EDA):** Service A just shouts out an *Event* (e.g., "Hey, a new post was created!") and goes back to its work. Service B hears this event whenever it's ready and acts on it.

2. Docker & Apache Kafka 🐋

To make EDA work, we need a "middleman" to hold all the shouted events. That's **Kafka**.

- **What is Kafka?** Think of it like a super-fast group chat or a mailroom. It stores events safely until services read them.
- **Why Docker?** Installing Kafka normally is a headache. Docker packages Kafka into a neat little box (container) so we can run it instantly with one command (`docker-compose up`).
- **What is KRaft Mode?** In the past, Kafka needed a "manager" program called **Zookeeper**. KRaft mode is the newer, better version where Kafka manages itself. No Zookeeper needed!

3. The Producer-Consumer Model 🤹

- 📩 **Producer:** Our code that checks for due posts and *produces* (sends) an event to the Kafka topic.
- 📩 **Consumer:** Our code that constantly listens to the topic and *consumes* (processes) the event to actually publish the post.

4. Making the System "Bulletproof" (Reliability) 🛡️

- **A. Retry Strategy & Exponential Backoff:** If publishing fails, don't give up instantly. Try again! *Exponential backoff* means waiting longer each time (1s, 2s, 4s) so we don't crash the system.
- **B. Dead Letter Queue (DLQ):** If a post fails to publish 3 times in a row, we throw it into a special Kafka topic called the DLQ. It's an isolation ward for broken events, so they don't block the good ones.
- **C. Idempotency:** Kafka might accidentally send the *same* event twice. To prevent publishing a post twice, our code checks a database (`ProcessedEventRepository`) to see if it already handled this event ID.

Part 2: Implementation Steps & Guide 🛠️

How do we actually code this? Here is the step-by-step game plan:

Step 1: Set up Kafka using Docker

- We create a `docker-compose.yml` file using the `apache/kafka:latest` image.

- We configure it to run in **KRaft Mode** by setting environment variables like `KAFKA_PROCESS_ROLES: broker,controller`. No separate Zookeeper container is needed!

Step 2: Configure the Spring Boot Backend

- Add dependencies for `spring-kafka`, `spring-boot-starter-web`, and `spring-boot-starter-data-jpa`.
- In `application.properties`, tell Spring Boot where Kafka is running (`localhost:9092`).

Step 3: Build the Producer (The Shouter)

- Create a `@Scheduled` method in Spring Boot that runs every 10 seconds.
- It asks the database: "Are there any posts scheduled for right now?"
- For each post, use `KafkaTemplate` to send a message to the `"post-topic"`.

Step 4: Build the Consumer (The Listener)

- Create a method annotated with `@KafkaListener(topics = "post-topic")`.
- When an event arrives, **first check for Idempotency**: "Does this event ID exist in my `ProcessedEvent` database table?" If yes, stop.
- Wrap the publishing logic in a `try/catch` block.
- **On Success:** Mark the post as `PUBLISHED` and save the event ID in the `ProcessedEvent` table.
- **On Failure:** If it fails completely, catch the error and use `KafkaTemplate` to send the exact same event to the `"post-topic-dlq"` (Dead Letter Queue).

Part 3: Production Level Examples 🌍

How do huge companies use this exact setup?

Example 1: Netflix (Video Encoding)

When you upload a new movie to Netflix, they don't just process it synchronously on one server.

- **Producer:** The upload server creates an event: "New Video Uploaded (ID: 123)".
- **Consumers:** Hundreds of different servers listen to this event. One server starts generating subtitles, another compresses the video for 4K, another compresses it for mobile.
- **DLQ:** If a specific video file is corrupted and crashes the compression server 3 times, it gets sent to a DLQ so an engineer can manually inspect the weird file.

Example 2: Uber / Food Delivery (Payment & Dispatch)

When you order food, multiple things need to happen: charge your card, tell the restaurant, find a driver.

- **Producer:** You click "Place Order". The app sends an "Order Placed" event to Kafka.
- **Consumers:** The Payment Service charges your card. The Restaurant Service prints the ticket. The Driver Service searches for a car.
- **Idempotency:** What if you had bad Wi-Fi and clicked "Place Order" twice by accident? Uber uses **Idempotency**. The Payment Service sees the exact same Order ID and says, "I already charged this card!" and ignores the duplicate click.